

Lecture 18 Notes

Opening: Continuing Block LDU

The lecture resumes the block matrix factorization started at the end of L17. The instructor stresses this material “leads to a key identity in many signal processing algorithms” and asks for patience: “after some algebra we will be all happy.” The plan: take a block-partitioned invertible matrix, perform **block Gaussian elimination** (block LDU), extract the **Schur complement**, the **determinant** and **inverse** formulas, derive the **Matrix Inversion Lemma (Woodbury identity)**, apply it to **recursive least squares**, and finally specialize to **Hermitian** matrices to connect to **inertia**, **positive definiteness**, and **linear matrix inequalities**.

Setup

Partition an invertible matrix A into four blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

with A_{11} of size $m \times m$, A_{22} of size $n \times n$. Assume A is invertible and A_{11} (and A_{22}) invertible. The goal is to convert A into block **diagonal** form using a block row operation on the left and a block column operation on the right.

Block LDU Factorization

Step 1 — Eliminate the Lower-Left Block (block row operation)

To make the $(2, 1)$ block zero **without touching the first block row**, left-multiply by a block **lower triangular** matrix:

$$\begin{bmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}.$$

The first block row is preserved (multiplied by I); the new $(2, 1)$ block is $-A_{21}A_{11}^{-1}A_{11} + A_{21} = 0$; and the new $(2, 2)$ block picks up the correction $-A_{21}A_{11}^{-1}A_{12}$. This is exactly Gaussian elimination at the block level — “multiply from the left by a lower triangular matrix to get an upper triangular form.”

Step 2 — Eliminate the Upper-Right Block (block column operation)

Now right-multiply the block upper triangular result by a block **upper triangular** matrix to clear the $(1, 2)$ block, **without touching the first block column**:

$$\begin{bmatrix} A_{11} & A_{12} \\ 0 & S_{11} \end{bmatrix} \begin{bmatrix} I & -A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix} = \begin{bmatrix} A_{11} & 0 \\ 0 & S_{11} \end{bmatrix},$$

where $S_{11} = A_{22} - A_{21}A_{11}^{-1}A_{12}$. The first block column is preserved; the $(1, 2)$ block becomes $-A_{11}A_{11}^{-1}A_{12} + A_{12} = 0$; and the $(2, 2)$ block is unaffected because it is multiplied by the zero block. We have reached **block diagonal** form by sandwiching A between a block lower triangular matrix (left) and a block upper triangular matrix (right).

Reconstruction

The triangular factors are easy to invert — just **flip the sign** of the off-diagonal block (you can verify this). Moving them to the right side:

$$A = \underbrace{\begin{bmatrix} I & 0 \\ A_{21}A_{11}^{-1} & I \end{bmatrix}}_{\text{block } L} \underbrace{\begin{bmatrix} A_{11} & 0 \\ 0 & S_{11} \end{bmatrix}}_{\text{block } D} \underbrace{\begin{bmatrix} I & A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix}}_{\text{block } U}.$$

This is the **block LDU factorization** — “diagonalization à la Gaussian elimination, applied at the block level.” The instructor cautions to be careful about block **sizes and order**, since matrix multiplication is not commutative.

The Schur Complement of A_{11}

The crucial new quantity is the **Schur complement** of A_{11} :

$$S_{11} = A_{22} - A_{21}A_{11}^{-1}A_{12}.$$

It is the “other corner” A_{22} with a correction subtracted off. **Memory aid** (the instructor’s): start at index **2-2**, transition to **2-1**, take the inverse of **1-1**, transition back from **1 to 2** (i.e., A_{12}) — giving $A_{22} - A_{21}A_{11}^{-1}A_{12}$.

Determinant Formula (Application 1)

Because $\det(\text{product}) = \prod \det$, and the block triangular factors have determinant 1 (triangular with identity diagonal blocks), while the block diagonal factor has determinant equal to the product of its diagonal blocks’ determinants:

$$\det(A) = \det(A_{11}) \cdot \det(S_{11}).$$

Intuition. If A were genuinely **block diagonal**, $\det(A) = \det(A_{11}) \det(A_{22})$. For a **non-diagonal** block matrix it is *not* $\det(A_{11}) \det(A_{22})$ but $\det(A_{11})$ times the determinant of the **Schur complement** of A_{11} . The dual (block UDL) gives $\det(A) = \det(A_{22}) \det(S_{22})$ with $S_{22} = A_{11} - A_{12}A_{22}^{-1}A_{21}$.

Scalar check. For $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the Schur complement of a is $d - ca^{-1}b$, and $a(d - ca^{-1}b) = ad - bc$ — the familiar 2×2 determinant.

Matrix Inverse via Block Factorization (Application 2)

Once $A = LDU$ (block), the inverse is easy because **each factor is easy to invert** — invert in **reverse order**:

$$A^{-1} = U^{-1}D^{-1}L^{-1}.$$

The block diagonal D inverts by inverting each diagonal block; the triangular factors invert by negating their off-diagonal block. Expanding gives the explicit **block inverse (from LDU)**:

$$A^{-1} = \begin{bmatrix} A_{11}^{-1} + A_{11}^{-1}A_{12}S_{11}^{-1}A_{21}A_{11}^{-1} & -A_{11}^{-1}A_{12}S_{11}^{-1} \\ -S_{11}^{-1}A_{21}A_{11}^{-1} & S_{11}^{-1} \end{bmatrix}.$$

Dual: Block UDL

Reversing the roles (eliminate A_{12} first using A_{22} as the pivot) yields a **block UDL** factorization with the **Schur complement of A_{22}** :

$$S_{22} = A_{11} - A_{12}A_{22}^{-1}A_{21},$$

the determinant $\det(A) = \det(A_{22}) \det(S_{22})$, and a second **block inverse (from UDL)**:

$$A^{-1} = \begin{bmatrix} S_{22}^{-1} & -S_{22}^{-1}A_{12}A_{22}^{-1} \\ -A_{22}^{-1}A_{21}S_{22}^{-1} & A_{22}^{-1} + A_{22}^{-1}A_{21}S_{22}^{-1}A_{12}A_{22}^{-1} \end{bmatrix}.$$

We now have **two equal but different-looking expressions** for A^{-1} .

Matrix Inversion Lemma (Woodbury Identity)

Equating the two alternative inverses (in particular their $(1, 1)$ blocks) and doing the algebra (which the instructor does not write out in full) yields a fundamental identity, called the **matrix inversion lemma** or **Woodbury identity**:

$$(A + BCD)^{-1} = A^{-1} - A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1}.$$

It inverts “a matrix plus a product of three matrices.” The result looks complicated but is enormously useful.

Instructor emphasis: “This is the only equation that I want you to memorize in this course.” It underlies adaptive signal processing, Bayesian estimation, and control.

Key use case. When A is large with a **known** inverse and BCD is a **low-rank** perturbation: instead of re-inverting the full $n \times n$ matrix, you invert only the small $(C^{-1} + DA^{-1}B)$. For a **rank-one** update (B a column, C a scalar, D a row), the only inverse needed is of a **scalar**.

Application: Sample Autocorrelation and Recursive Least Squares (RLS)

From Theory to Data

In estimation theory, optimal estimators are often built from the **autocorrelation matrix** $R_h = \mathbb{E}[hh^*]$ (the library of pairwise correlations among the entries of a random vector h). But in real applications **nobody hands you** R_h — you must estimate it from data.

Sample autocorrelation (analogy to the sample mean). Just as the mean $\mathbb{E}[x]$ is estimated from samples by averaging ($\frac{1}{N} \sum x_i$), the autocorrelation is estimated by averaging outer products of the observed sample vectors $h_1, \dots, h_N$:

$$\hat{R}^{(N)} = \frac{1}{N} \sum_{i=1}^N h_i h_i^*.$$

The instructor explicitly draws the parallel: we don’t have the expectation, but we have samples, so we average the $h_i h_i^*$ “auto-products.”

The Online Update Problem

In **online** applications data keeps arriving. When sample h_{N+1} appears:

$$\hat{R}^{(N+1)} = \frac{1}{N+1} \sum_{i=1}^{N+1} h_i h_i^* = \frac{N}{N+1} \hat{R}^{(N)} + \frac{1}{N+1} h_{N+1} h_{N+1}^*.$$

This is a **rank-one update** — adding the rank-one outer product $h_{N+1} h_{N+1}^*$ to a scaled copy of the previous estimate, so you reuse $\hat{R}^{(N)}$ instead of re-summing from scratch.

But many estimators need the **inverse** $\hat{R}^{-1}$. Re-inverting a (say) 1000×1000 matrix at every step costs $\sim n^3 \approx 10^9$ operations per sample — impractical for real-time use. The instructor’s idea: **update the inverse directly** rather than the matrix.

Applying Woodbury

The instructor rewrites the update by pulling out the scalar factor:

$$\hat{R}^{(N+1)} = \frac{N}{N+1} \left[\hat{R}^{(N)} + \frac{1}{N} h_{N+1} h_{N+1}^* \right].$$

Inside the brackets this is exactly the Woodbury form $A + BCD$ with

$$A = \hat{R}^{(N)}, \quad B = h_{N+1}, \quad C = \frac{1}{N} \text{ (scalar)}, \quad D = h_{N+1}^*.$$

Woodbury gives $(\hat{R}^{(N+1)})^{-1}$ entirely in terms of the **previous inverse** $(\hat{R}^{(N)})^{-1}$ and the new vector h_{N+1} :

$$(\hat{R}^{(N+1)})^{-1} = \frac{N+1}{N} \left[(\hat{R}^{(N)})^{-1} - \frac{(\hat{R}^{(N)})^{-1} h_{N+1} h_{N+1}^* (\hat{R}^{(N)})^{-1}}{N + h_{N+1}^* (\hat{R}^{(N)})^{-1} h_{N+1}} \right].$$

The denominator comes from $C^{-1} + DA^{-1}B = N + h_{N+1}^* (\hat{R}^{(N)})^{-1} h_{N+1}$, and it is a **scalar** (row $\times$ matrix $\times$ column = 1×1), so the only “inversion” is a scalar division. Everything else is matrix-vector products costing $O(n^2)$. **Net: $O(n^2)$ per update instead of $O(n^3)$.** This is the **Recursive Least Squares (RLS)** algorithm, central to adaptive filtering (adaptive equalizers, noise cancellation). The instructor notes RLS will be revisited when least squares is covered.

Hermitian Case: Star-Congruence and Inertia

Now specialize: let A be **Hermitian**. Then A_{11} and A_{22} are Hermitian and $A_{21} = A_{12}^*$. The same block LDU still applies (it works for any 2×2 block partition); the only change is $A_{21} \rightarrow A_{12}^*$. Crucially, the **left and right triangular factors are now conjugate transposes of each other**:

$$A = S \begin{bmatrix} A_{11} & 0 \\ 0 & S_{11} \end{bmatrix} S^*, \quad S = \begin{bmatrix} I & 0 \\ A_{12}^* A_{11}^{-1} & I \end{bmatrix}.$$

Since S is invertible, this is a **star-congruence** between A and the block diagonal matrix.

Inertia Additivity

By **Sylvester’s law of inertia** (star-congruence preserves inertia), and because the inertia of a block diagonal matrix is the **sum** of the inertias of its diagonal blocks:

$$\text{inertia}(A) = \text{inertia}(A_{11}) + \text{inertia}(S_{11}).$$

The number of positive eigenvalues of A equals the positive count of A_{11} plus that of S_{11} , and likewise for negative and zero counts. The dual UDL form gives $\text{inertia}(A) = \text{inertia}(A_{22}) + \text{inertia}(S_{22})$. The instructor calls this “very useful information in many control and estimation problems.”

Positive Definiteness Criterion

$$A \succ 0 \iff A_{11} \succ 0 \text{ and } S_{11} = A_{22} - A_{21} A_{11}^{-1} A_{12} \succ 0.$$

Proof. $A \succ 0$ iff its inertia is all-positive; by additivity that holds iff both A_{11} and S_{11} have all-positive inertia, i.e., both are PD.

Important warning (stated emphatically). Knowing $A_{11} \succ 0$ **and** $A_{22} \succ 0$ is **not** enough to conclude $A \succ 0$. You must check the Schur complement. The Schur complement is A_{22} **minus** a positive semidefinite matrix $A_{21} A_{11}^{-1} A_{12}$; subtracting a PSD matrix from a PD matrix can make it **indefinite**. So PD of the two diagonal corners is **necessary but not sufficient**; PD of one corner *and its Schur complement* is necessary **and** sufficient. (If $A_{12} = 0$, i.e., block diagonal, then $A \succ 0 \iff A_{11} \succ 0$ and $A_{22} \succ 0$.)

The Quiz: Why $A_{21} A_{11}^{-1} A_{12} \succeq 0$

The instructor poses this as a quiz, offering **two approaches**:

1. (*Eigenvalue route*) compute eigenvalues and show they are nonneg — harder.
2. (*Quadratic-form route, preferred*) multiply by a row vector on the left and its conjugate on the right. Write $D = A_{12}x$. Then

$$x^* (A_{12}^* A_{11}^{-1} A_{12}) x = (A_{12}x)^* A_{11}^{-1} (A_{12}x) = D^* A_{11}^{-1} D \geq 0,$$

because $A_{11} \succ 0$ implies $A_{11}^{-1} \succ 0$. It is 0 only when $D = A_{12}x = 0$. Hence $A_{12}^* A_{11}^{-1} A_{12} \succeq 0$ — always positive **semidefinite** — “no need to go into eigenvalue calculations.” This is exactly the PSD term that can drag A_{22} down into indefiniteness.

Linear Matrix Inequality (LMI) via Schur Complement

In applications the criterion is usually used in **reverse**: to handle a **quadratic** matrix inequality, convert it into a **linear** one. Suppose you must show a quadratic-in- H expression like

$$B - HC^{-1}H^* \succ 0 \quad (\text{quadratic in } H),$$

given $C \succ 0$. Via the Schur complement equivalence this is the same as a condition on a larger matrix that is **linear** in H :

$$\begin{bmatrix} C & H^* \\ H & B \end{bmatrix} \succ 0 \iff C \succ 0 \text{ and } B - HC^{-1}H^* \succ 0.$$

The sign and exact placement of the blocks depend on how the original quadratic inequality is written, but the trick is always the same: the quadratic product in H becomes an off-diagonal block where H appears only linearly. The big matrix is called a **linear matrix inequality (LMI)**. **Why this matters:** LMI constraints are **convex**, so problems with LMI constraints are solvable by **semidefinite programming (SDP)**. “You can convert quadratic optimization problems into linear (matrix-inequality) forms using this Schur complement trick” — a standard technique in control theory (Lyapunov stability, H_∞ control).

The instructor summarizes the arc: “we just started from Gaussian elimination” of a block-partitioned matrix, did LDU and UDL, obtained two inverse forms ($\rightarrow$ Woodbury), and in the Hermitian case got star-congruence and inertia results ($\rightarrow$ PD criteria, LMIs).

Transition to Normed Vector Spaces

The next major topic is **normed spaces** — extending the norm concept **beyond the Euclidean norm**, first for n -dimensional vectors and later to arbitrary vector spaces (matrices, functions), each useful in different applications.

A norm is an added structure. The Euclidean norm is **not** part of the definition of a vector space — a vector space is four objects (set of vectors, set of scalars, vector addition, scalar multiplication). A **normed vector space** adds a fifth object: a **norm** that measures how big each vector is. Different choices of that fifth object give different geometries.

Motivation: Combining Several Numbers into One

Take the two-dimensional real vector space; a vector like $(3, 4)$ is a point in the plane (its position). To say “how big” it is, you must combine its several numbers into a **single** number. The Euclidean norm does this by measuring **distance to the origin** — the hypotenuse $\sqrt{3^2 + 4^2} = 5$. (The instructor mentions the hand-drawn “norm” figure his daughter made during Covid, which he still keeps because she likes it.)

But this is just **one** of infinitely many ways to measure size, even in 2-D.

The Taxicab (Manhattan, ℓ_1) Norm

Imagine traveling from the origin to $(3, 4)$: - A **helicopter** can fly the straight Euclidean path: distance 5. - A **pedestrian or taxi** in Manhattan must follow the grid of horizontal and vertical streets (avoiding “Broadway Avenue,” the one strange diagonal road — the instructor jokes a Turkish migrant must have disrupted the perfect grid). Either route covers 3 horizontal + 4 vertical = 7, regardless of which coffee shops you pass (as long as you don’t backtrack).

So the **taxicab / Manhattan / ℓ_1 norm** of $(3, 4)$ is the **sum of absolute values**:

$$\|x\|_1 = \sum_i |x_i| = 3 + 4 = 7.$$

Which is “correct”? Neither — it depends on the application (helicopter $\rightarrow$ Euclidean; taxi $\rightarrow \ell_1$). Note $\|x\|_1 = 7 \geq \|x\|_2 = 5$: the ℓ_1 -norm is always $\geq$ the Euclidean norm. The instructor insists on the Manhattan map because of this “minutes/Manhattan” property.

Why ℓ_1 Became Important: Sparsity

Historically the Euclidean norm dominated 20th-century engineering (people wrote $\|\cdot\|$ and everyone assumed Euclidean). After the **1990s**, the ℓ_1 -norm surged in importance — **not** because of Manhattan travel, but because of **sparse representations**.

The homework problem (and a historically important problem). Consider an **underdetermined** system $Ax = b$ with A **fat** (infinitely many solutions). Among all solutions, pick the one of **minimum norm**. The norm you choose shapes the solution: - Minimizing the ℓ_1 -**norm** subject to $Ax = b$ tends to yield the **sparsest** solution — the x with the **maximum number of zeros** (under suitable conditions).

The truly “right” objective — minimize the **number of nonzero entries** (cardinality, the “ ℓ_0 ” count) — is a **hard, non-convex** problem. Remarkably, minimizing the **convex** ℓ_1 -norm is, under certain conditions, **equivalent** to that hard problem. This non-obvious finding was a **turning point in the 1990s**, spawning enormous literature (“hundreds of thousands of papers”). A vector that is mostly zeros is called **sparse**, and this is why ℓ_1 is so important.

Modern uses: ℓ_1 regularization of neural network weights forces most weights toward zero (a form of regularization); applied to a layer’s **activations**, it forces most activations to zero — echoing the **brain**, where of approximately 80–100 billion neurons only small groups fire at any time (sparse activation). The instructor recommends an “ ℓ_1 -norm magic” video for those who want to explore why ℓ_1 produces sparsity.

Tooling (homework logistics). The last homework problem is computer-oriented: download **CVX**, the convex optimization package from **Stephen Boyd’s** group at Stanford (originally MATLAB, now also Python; **Julia** is another option — the instructor tried Julia, hit bugs, and went back to Python). He strongly recommends taking an **advanced optimization** course after EE 545 and points to Boyd’s YouTube lectures. He predicts students will end up most grateful for learning CVX.

Norm Axioms

Can *any* scalar-valued function of a vector be a norm? No — it must “measure bigness.” Formally, a **norm** is a real-valued function $f : V \rightarrow \mathbb{R}$ on a vector space satisfying:

1. **Non-negativity / definiteness:** $\|x\| \geq 0$, and $\|x\| = 0$ **iff** $x = 0$ (a nonzero vector must have positive norm).
2. **Homogeneity:** $\|\alpha x\| = |\alpha| \|x\|$ — scaling the vector scales its norm by the **absolute value** of the scalar.
3. **Triangle inequality:** $\|x + y\| \leq \|x\| + \|y\|$ — the third side of a triangle is no longer than the sum of the other two.

This is the **positive-definite norm** definition the course adopts.

Aside: Minkowski Functional / Non-Standard “Norms”

The instructor notes you *can* build “weird geometries” that violate these rules. Example: in **relativity**, a spacetime position (x, y, z, t) uses the **Minkowski functional**, where the associated expression can be **zero even when** $(x, y, z, t) \neq 0$ (violating definiteness). Such Minkowski/Lorentzian “norm” spaces appear in EE too — e.g., in H_∞ robust control, some formulations use spaces not governed by the standard norm axioms. The course, however, sticks with the standard positive-definite norms.

The p -Norm Family

Both the Euclidean and taxicab norms are special cases of the **p -norm**. Writing the Euclidean norm as $\|x\|_2 = \sqrt{x^*x} = (\sum_i |x_i|^2)^{1/2}$ (absolute values handle complex entries), generalize to

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{1/p}.$$

- $p = 1$: taxicab / ℓ_1 norm (sum of absolute values). - $p = 2$: Euclidean / ℓ_2 norm. - $p \rightarrow \infty$: the ℓ_∞ (Chebyshev / max) norm — it picks out the **maximum absolute element**:

$$\|x\|_\infty = \max_i |x_i|.$$

For example, $\|(3, -4)\|_\infty = 4$. The p -norms are the most famous family, but **not the only useful norms** — the course will introduce others on $\mathbb{C}^n$ (and on matrices and functions). “All norms are useful, depending on the application.”

The instructor stops here, leaving the detailed norm development to be picked up in the next lecture.

Instructor Remarks and Study Guidance

- The **Schur complement** $S_{11} = A_{22} - A_{21}A_{11}^{-1}A_{12}$ is one of the most important expressions in applied linear algebra — memorize its form (2-2, 2-1, invert 1-1, 1-2).
- $\det(A) = \det(A_{11}) \det(S_{11})$ generalizes $ad - bc$; a non-diagonal block matrix uses the **Schur complement**, not $\det(A_{22})$.
- The **Woodbury / matrix inversion lemma** is “the only equation to memorize.” Its killer app is **rank-one (or low-rank) updates of an inverse** — the basis of **RLS** ($O(n^2)$ per step vs. $O(n^3)$).
- For **Hermitian** A : $A \succ 0$ iff $A_{11} \succ 0$ **and** the Schur complement $S_{11} \succ 0$. Both corners being PD is necessary but **not sufficient**.
- $A_{12}^* A_{11}^{-1} A_{12} \succeq 0$ is shown cleanly by the quadratic-form trick, not eigenvalues.
- The Schur complement converts **quadratic** matrix inequalities into **linear** matrix inequalities (LMIs), which are convex and SDP-solvable — key in control.
- A **norm** is an added structure on a vector space satisfying non-negativity/definiteness, homogeneity, and the triangle inequality. The ℓ_1 (taxicab) norm $= \sum |x_i| \geq \|x\|_2$; minimizing ℓ_1 promotes **sparsity** (1990s breakthrough, neural-net/brain analogy). $\ell_\infty = \max_i |x_i|$. Do the CVX homework early.

Source and Coverage Note

Source: `corrected/lecture18_corrected.md`.

Coverage: Block LDU factorization (block Gaussian elimination, both elimination steps, reconstruction); Schur complement of A_{11} and memory aid; determinant formula and block-diagonal-vs-not intuition with scalar check; block inverse via reverse-order inversion (LDU and UDL forms); block UDL dual and Schur complement of A_{22} ; equating the two inverses to obtain the Matrix Inversion Lemma (Woodbury identity) and its low-rank use case; sample autocorrelation matrix (sample-mean analogy), online rank-one update, RLS derivation via Woodbury ($O(n^2)$ vs $O(n^3)$); Hermitian case star-congruence and inertia additivity (LDU and UDL); positive-definiteness criterion with the emphatic “necessary but not sufficient” warning; the instructor’s quiz showing $A_{12}^* A_{11}^{-1} A_{12} \succeq 0$ via the quadratic-form trick; LMI via Schur complement (quadratic $\rightarrow$ linear, convexity, SDP, control); full introduction to normed vector spaces (norm as added structure, motivation, taxicab/Manhattan/ ℓ_1 norm with helicopter-vs-pedestrian and the daughter’s-map anecdote, 5 vs 7 and $\ell_1 \geq \ell_2$, sparsity history and the underdetermined min-norm/ ℓ_0 -vs- ℓ_1 story, CVX/Boyd homework and optimization-course recommendation, neural-net/brain sparsity, norm axioms, Minkowski-functional/relativity/ H_∞ aside, the p -norm family and $\ell_\infty = \max |x_i|$).